

Project 3 – Final Retrospective & Presentation Preparation

Section 1 – Sprint 6 Outcome

Sprint 6 Goal

(Copy exactly from your Sprint 6 Planning Document)

Sprint Goal:

By the end of Sprint 6, a user will be able to repeatedly execute Momentum's full end-to-end flow (log in, browse and filter trails by difficulty and surface type, open and close a trail detail modal, log a ride to the activity calendar, and view CO2 and money savings).

Sprint Goal Achievement

Fully Met

Evidence

What evidence shows whether the Sprint Goal was met? Include at least one specific piece of evidence.

(Reference demo behavior, completed functionality, GitHub activity, testing results, or other observable evidence.)

At the Sprint 6 Demo, the full flow ran cleanly on a logged-out browser with no refreshes, resets, or verbal explanations required. The three Sprint 6 bug fixes (Trail Browsing filter useEffect, modal outside-click close handler, and Calculator money savings bar scaling) were merged to main and each passed our team's two-pass walkthrough across two separate class periods. On GitHub, all three fix tickets were closed against the Sprint 6 milestone, and the filter bug that broke the 5/15 Sprint 5 demo did not reappear.

Final Sprint Outcome

In ONE sentence, what can a user do now because of Sprint 6?

A user can now log into Momentum, browse and filter the trail list by difficulty and surface type across consecutive filter changes without stale cards, open a trail detail modal and close it via clicking outside the modal, log a ride to the activity calendar, and view CO2 and money savings in the Calculator.

Release Readiness Evaluation

At the end of Sprint 6, how ready was your product for final presentation?

Fully Ready

Explain:

Project 3 – Final Retrospective & Presentation Preparation

By the Sprint 6 Demo, our team's core flow was stable and repeatable, and the bug that sank our Sprint 5 demo was fixed and re-verified. The only thing that was short of fully ready was the Trail Browsing filter under rapid, repeated input. The useEffect refactor fixed the normal case, but we had not exhaustively tested every fast filter-switching sequence, so for that purpose, we kept a clean-browser walkthrough as a safeguard.

Section 2 – Product Evolution

Starting Point

What could a user do at the end of Sprint 1 that could be demonstrated during the Sprint 1 Review?
(Be specific.)

At the Sprint 1 Review, a user could open Momentum and move through the core page structure (navbar and dynamic pages) with the frontend connected to the Supabase database. There was no working trail filtering, activity log, or savings calculation yet. The demo showed the app shell and the live database connection worked, not full feature behavior.

Current State

What can a user do today?
(Be specific.)

Today, a user can log in through Supabase authentication, browse and filter trails by difficulty and surface type, open and close a trail detail modal, log a ride to the activity calendar, view their CO2 and money savings in the Calculator, upload images and documents to a personal gallery (that persists), find a biking route (based on user inputs) on a map, and view a calendar with biking events (and sign up for those events). The full flow runs end to end in a single session and holds up when repeated.

Most Important Product Milestones

Identify the three most important milestones in your product's development. Milestones may include MVP completion, major feature additions, architecture improvements, performance improvements, or major bug fixes.

Sprint	Milestone	Why It Mattered
Sprint 1	Core app structure connected to Supabase (navbar, dynamic pages, live database)	This was the foundation everything else was built on. A working shell plus a real database connection let us add actual features instead of fighting setup later.
Sprint 4	CO2 and money savings calculation	This is what gives Momentum its purpose. It turned the app from a trail browser into

Project 3 – Final Retrospective & Presentation Preparation

		something that shows the real environmental and money impact of biking, which is the whole point of the product.
Sprint 6	Stable, repeatable end-to-end flow	After the Sprint 5 demo failed on a repeat run, fixing the filter, modal, and calculator bugs made the full flow work reliably twice in a row. That is what made the product presentable.

Evidence of Product Growth

What evidence best demonstrates how much your product improved over the semester?
(Check all that apply)

- ☒ Demo history
- ☒ GitHub activity
- ☐ Product screenshots
- ☒ User workflow expansion
- ☐ Performance improvements
- ☒ Stability improvements
- ☒ UI/UX improvements
- ☒ Testing improvements
- ☐ Other: _____

Explain:

The clearest evidence is the demo history. At the Sprint 1 Review the demo was just the app shell and a database connection, and by the Sprint 6 Review a user could run the full log-in-to-savings flow twice in a row without a reset. GitHub backs this up with 47 closed issues across the semester. The biggest jump was in stability and testing: after the Sprint 5 filter bug broke a repeat run, the two-pass walkthrough rule caught regressions before they reached a demo.

Section 3 – Scrum System Evolution

Biggest Recurring Problem

What issue appeared most often throughout the project?

Testing too late

Describe how this issue affected your team.

Project 3 – Final Retrospective & Presentation Preparation

We kept verifying features too late and only once, usually right before a demo and in a single pass. Bugs that only showed up on a second run, like the Trail Browsing filter, slipped through. It directly caused the Sprint 5 demo to fail on 5/15 when the filter broke on a repeat run in front of the class.

Most Effective Improvement

What process change had the biggest positive impact on your team?

Switching from a single-pass walkthrough to a two-consecutive-run walkthrough on a projected clean browser, owned by one person (Aarush as Integration Verifier) during the final 15 minutes of every class. Anything that failed the second run was reverted from main before class ended.

Evidence It Worked

What evidence shows this improvement helped?

(Reference demo outcomes, board behavior, GitHub workflow, team performance, etc.)

The Sprint 6 demo ran the full flow twice in a row with no refresh, which is the exact thing that failed in Sprint 5. The filter bug never came back once features had to survive a second run before being kept on main. Having one named owner and a fixed time meant the check actually happened instead of being skipped.

Most Effective Retrospective Rule

What retrospective rule produced the most meaningful improvement?

Rule:

Aarush runs a two-consecutive-run walkthrough of the full flow on a projected clean browser in the final 15 minutes of every class, and any feature that fails the second run is reverted from main before class ends.

Why it worked:

It fixed both problems behind the Sprint 5 failure at once. The second run caught repeatability bugs a single pass could never see, and pinning it to a specific person, a specific time, and a clear consequence meant it got enforced instead of drifting like the old rule did.

Least Effective Retrospective Rule

What rule had little impact or was difficult to enforce?

Rule:

Project 3 – Final Retrospective & Presentation Preparation

The earlier Sprint 4 rule: do a single-pass walkthrough in the final 10 minutes of every class period.

Why it was ineffective:

The timing drifted in practice to a few minutes after class, so it was not always done, and a single pass could not catch bugs that only appeared on a second run. It gave a false sense that features worked, which is exactly how the filter bug reached the Sprint 5 demo.

Section 4 – Team Growth

Technical Growth

What is the most important technical skill your team developed?

The most important technical skill our team developed was managing React state cleanly across features that share it, especially getting `useEffect` dependency arrays right so that one feature's re-render does not break another.

Evidence:

The Sprint 5 filter bug came from a bad dependency in the Trail Browsing `useEffect` that left stale cards on screen. Fixing this issue during Sprint 6 by depending on the raw difficulty and surface-type state, plus a pre-merge check on the Activity Log that shared that state, showed we understood the shared-state problem instead of just patching the symptom.

Process Growth

What is the most important Scrum or project-management skill your team developed?

The most important Scrum/project-management skill that we developed was writing enforceable process rules with a clear owner, time, and consequence instead of vague good intentions.

Evidence:

Our team's Sprint 4 rule was really vague on timing and ownership and quietly stopped happening. The Sprint 6 rule named Aarush specifically, set the final 15 minutes of class, and defined revert-before-class as the consequence. That version of the rule actually worked every class.

Collaboration Growth

How did your team's collaboration improve during the project?

Project 3 – Final Retrospective & Presentation Preparation

Our team moved from solo development work (with little collaboration) to shared ownership of the main branch and paired programming, where no one would merge something that the rest of the team could not verify.

Evidence:

Early on, we paired less experienced members (Sriram and Aarush) with stronger React devs (Sath and Yohaán) to spread the framework knowledge. By Sprint 6, the two-pass walkthrough meant that the whole team watched features get verified together before they merged to main, so problems were caught as a group instead of being discovered at the demo in front of a live audience.

Adaptation Example

Describe one situation where your team encountered a significant challenge and adapted its plan, workflow, or product to overcome it.

What happened?

The Sprint 5 demo on 05/15 failed. The Trail Browsing filter worked on the first run but left stale trail cards when the filter was changed a second time, and it broke in front of the class. Because majority of our team was absent on 05/13 and 05/14 (because of AP Physics C Exams), we did not have any buffer to catch the issue.

How did the team respond?

In the Sprint 5 retrospective, we traced the issue to its root causes instead of blaming the demo itself. A single-pass walkthrough could never catch a second-run bug, and the verification timing had drifted. Because of that, we rewrote the retrospective rule to require two consecutive runs on a clean browser, gave the rule a named owner, and required failing features to be reverted before the period ended.

What was the result?

In Sprint 6, the same flow ran twice in a row cleanly at the review, and the filter bug did not reappear during our demo. The change that we made turned the worst moment of the project into a fix that made our final demo stable.

Section 5 – Lessons Learned

Biggest Success

What was your team's greatest success during this project?

Project 3 – Final Retrospective & Presentation Preparation

Recovering from the failed Sprint 5 demo and shipping a full end-to-end flow that was stable and repeatable by the final presentation.

Evidence:

The Sprint 6 demo ran the complete log-in-to-savings flow twice in a row on a clean browser with no refresh, and all three bug fixes were merged and verified across two class periods.

Biggest Mistake

What was your team's biggest mistake?

Trusting a single-pass walkthrough as proof that a feature worked, and letting the verification timing drift instead of pinning it down.

What did you learn from it?

That a feature is not done just because it worked once. Repeatability has to be tested directly, and a rule only helps if it has a clear owner, time, and consequence. Otherwise it quietly stops happening right when you need it most.

Missed Opportunity

What is one thing you wish your team had done differently earlier?

Setting up the two-pass, clean-browser verification earlier in the project. If we had been testing repeatability from Sprint 3 or 4, the Sprint 5 demo failure probably never happens.

Advice to Future Teams

If next year's students were starting this project tomorrow, what are the three most important pieces of advice you would give them?

Advice #1: Test every feature twice in a row on a new browser before you mark it done. Most demo-day bugs only show up on the second run.

Advice #2: Make your process rules specific. Name who enforces it, exactly when, and what happens when something fails. Vague rules drift and stop happening.

Advice #3: When testing, don't go in expecting success. Look for errors and check edge cases because if you don't do it earlier, it will appear on demo day.

Section 6 – Presentation Preparation

Key Milestones to Highlight

Project 3 – Final Retrospective & Presentation Preparation

Identify 3–5 moments that should appear in your final presentation.

Sprint	Event/Milestone	Why It Matters
Sprint 1	App shell connected to Supabase	Sets the baseline so the audience can see how far the product came.
Sprint 4	CO2 and money savings calculation	The feature that gives Momentum its purpose and is the core of the live demo.
Sprint 5	Demo failure on the trail filter (5/15)	The honest low point that sets up the recovery story and shows real problem solving.
Sprint 6	Two-pass verification rule plus the three bug fixes	The process change and fixes that turned the failure around.
Sprint 6	Stable end-to-end flow at the final review	The payoff: the full flow running twice in a row with no reset.

Evidence of Growth

What evidence best demonstrates your growth over the semester?

(Check all that apply)

- ☒ Product functionality
- ☒ Product screenshots/videos
- ☒ GitHub activity
- ☐ Sprint board history
- ☒ Demo improvements
- ☒ Process improvements identified through retrospectives
- ☒ Team workflow improvements
- ☐ Other: _____

Explain:

Our strongest evidence is the contrast in demo behavior. The Sprint 5 demo broke on a repeat run and the Sprint 6 demo ran the full flow twice cleanly. That improvement came straight from the retrospective: we identified the single-pass testing problem and built a two-pass verification rule the whole team followed. GitHub closed issues and the working product back it up.

Challenges Overcome

Identify the two biggest challenges your team successfully overcame.

Project 3 – Final Retrospective & Presentation Preparation

Challenge 1:

Our first challenge was that the Sprint 5 demo failed in front of the whole class when the Trail Browsing filter left stale cards on a repeat run in front of the class.

How you overcame it:

We overcame the challenge by tracing the root cause back to a bad useEffect dependency and to a verification process that only tested features once. We fixed the dependency to track the raw filter state and replaced the single-pass check with a two-consecutive-run walkthrough on a clean browser.

Challenge 2:

Our second challenge was that our process rules that did not stick. Our Sprint 4 verification rule had vague timing and no clear owner, so it quietly stopped being done.

How you overcame it:

We rewrote the rule with a named owner (Aarush), a fixed time (final 15 minutes of class), and a clear consequence (revert failing features before class ends). That version actually ran every class.

Audience Takeaway

After seeing your presentation, what is the ONE thing you want the audience to remember?

That regardless of the difficulties we faced in demo, no matter how large the scope we challenge ourselves with was, we managed to preserve and fully tell our story.

Presentation Evidence Planning

What three pieces of evidence are most important to include in your final presentation?

Evidence 1:

Side-by-side demo most likely. having the Sprint 5 filter failure explained and documented versus the Sprint 6 running twice with no errors.

Evidence 2:

The retrospective change: the old single pass rule next to the new two-pass, clean-browser rule with its owner, time, and consequence.

Evidence 3:

Project 3 – Final Retrospective & Presentation Preparation

GitHub history showing 47 closed issues across the semester and the three Sprint 6 bug-fix tickets closed against the milestone.

Why were these selected?

Together they tell where we broke, what we changed in our process, and proof in the product and the repo that the change worked. They focus on growth and problem solving, not just the final feature list.

Section 7 – Final Reflection

What are you most proud of?

Turning the Sprint 5 demo failure into the fix that made the final product stable. We did not just patch the bug; we changed how we verified work so the same kind of failure could not happen again. We made it so that the dates would no longer clash, we also made it so that map would consistently function under our school Wi-Fi.

What would you still improve if you had one more Sprint?

I would improve the Trail Browsing filter against the repeated input edge cases we never fully tested such as combinations and spam, and add a small, automated test so the repeatability check did not depend on a person remembering to run it.

Looking Back

Complete this sentence:

The most important thing our team learned from this project was:

That testing a feature once is not proof it works, and that a process rule only helps if it has a clear owner, a specific time, and a real consequence.

Final Project Evaluation

Overall, how successful was your project?

Successful

Explain your reasoning using evidence from the project.

Project 3 – Final Retrospective & Presentation Preparation

We met our final Sprint goal: a user can run Momentum's full flow from log in to CO2 and money savings, and it held up twice in a row at the Sprint 6 review. We also recovered from a demo failure by improving our process, which is a stronger outcome than if nothing had gone wrong. It is not rated higher because we hit a public demo failure in Sprint 5 and left some edge cases untested, but the product worked reliably by the end, and the team clearly grew in knowledge. We all picked up skills related to Github and understanding of React.